

Image Processing - Lecture 4



Amir Atapour-Abarghouei

amir.atapour-abarghouei@durham.ac.uk

Reading:

Szeliski – Section 3.3.1

Gonzalez/Woods – Section 2.5 + 5.3

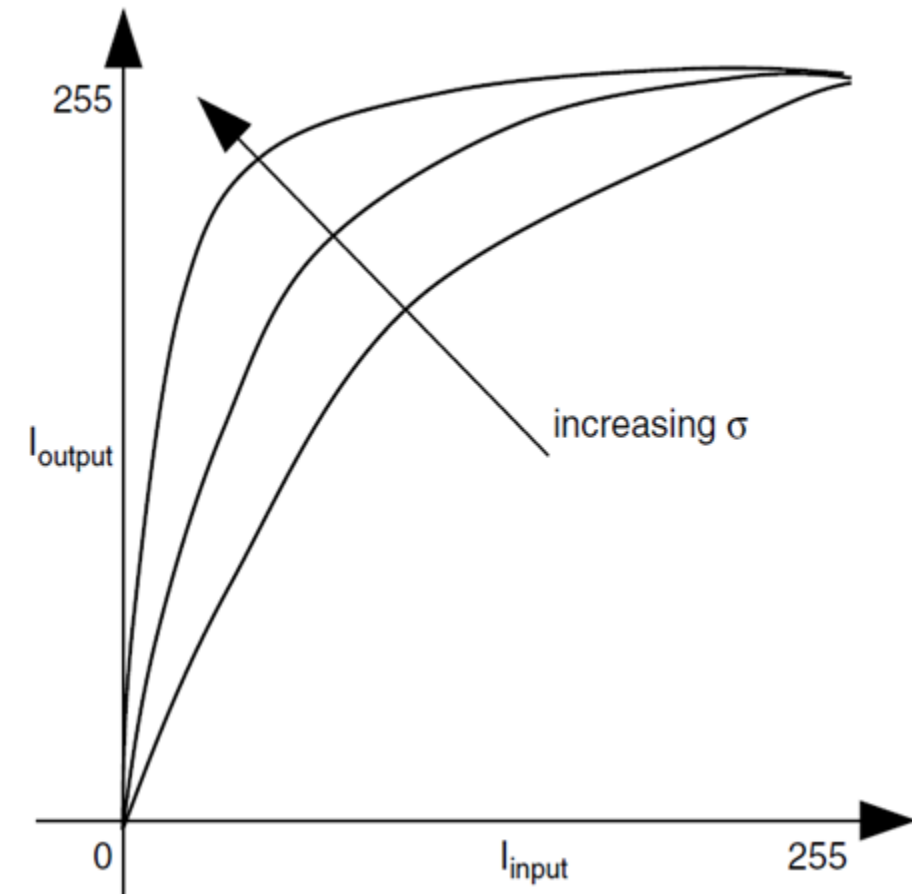
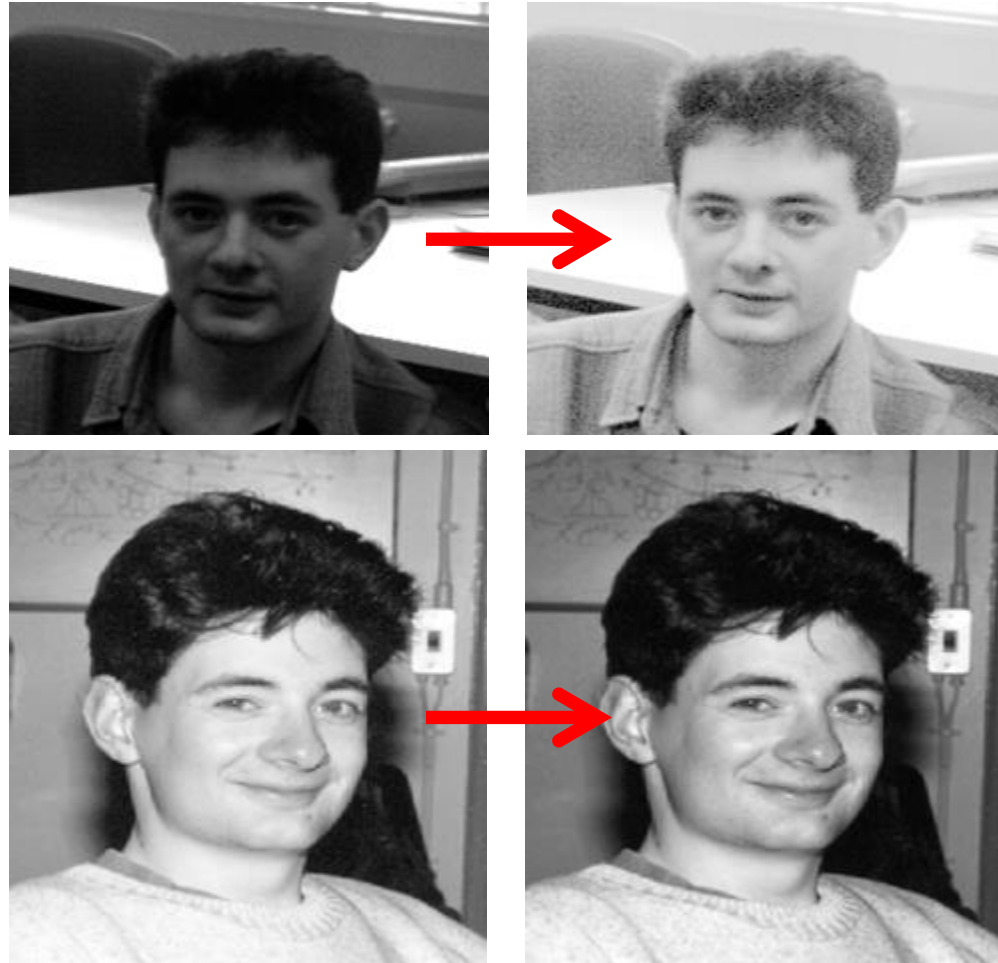
Solomon/Breckon – Ch. 4.1 – 4.4

Image Noise and Spatial Filtering I

Slide material acknowledgements: Fisher; Breckon; Ivrisimtzis; Atapour

Recap ... Point Transforms

(simple, but efficient & powerful)



Previously: Intensity transforms for contrast enhancement

Today: image noise and how
we can remove it!

Noise in Images

No digital image is a perfect representation of the original 2D signal.

- They are limited in resolution by sampling and contain **noise**.

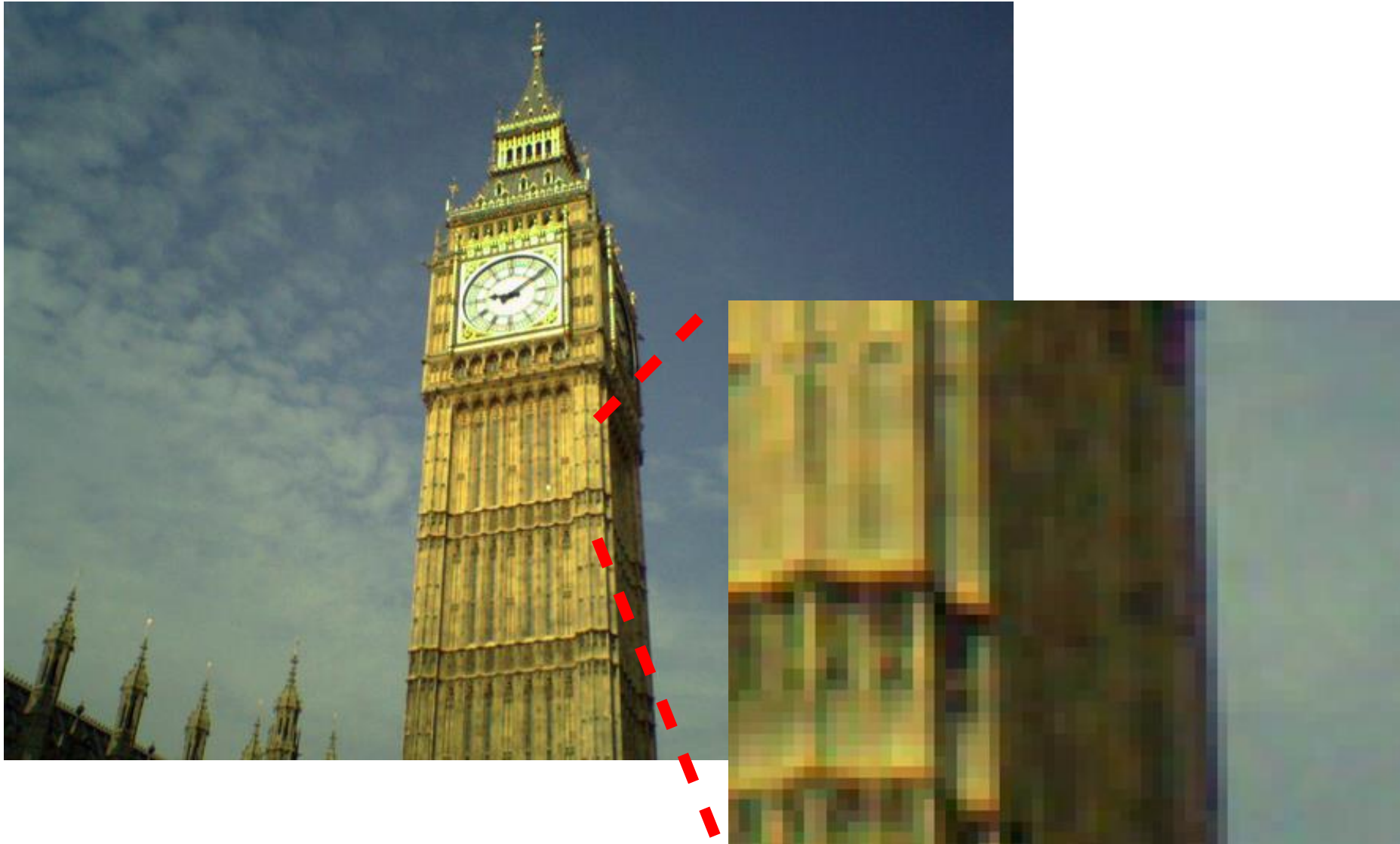
Noise removal is a major goal of image processing.

- Why? To limit its effects on image visualisation and analysis.

If images were noise free, then life would be easy.

- They are not, so it is not.
- *and hence big companies employ lots of software engineers in the global software industry!*

Image = Information + Noise

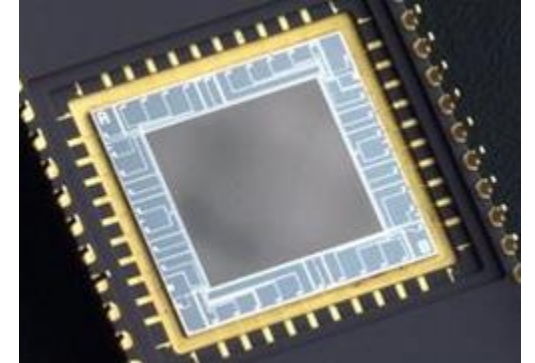


Noise is random variations in pixel values not directly attributable to scene contents.

Where does image noise come from?

Capture (in the camera itself):

- variations in sensor temperature
- electrical sensor noise
- sensor non-uniformity
- dust in the environment
- vibration
- lens distortion
- focus limitations
- sensor saturation (too much light)
- under-exposure (too little light)



CCD photo sensor

Where does image noise come from?

Sampling:

- limitations in sampling and intensity quantisation (aliasing) when transforming a continuous signal to a digital one.

Processing:

- limitations in numerical precision (floating point numbers).
- potential integer overflow.
- mathematical approximations (e.g. $\pi = 3.14\dots$).

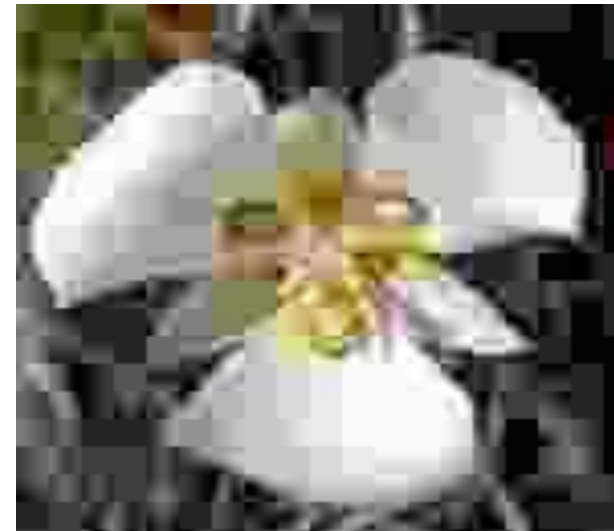
Where does image noise come from?

Image Compression:

- **lossy** techniques remove information from image to reduce file size.
- JPEG / MPEG are examples of widely used lossy compression formats.
- remove non-perceivable detail aiming at “no noticeable difference”.
- **result**: “compression artefacts” in the image.



JPEG: 42KB



12KB

Several formats, such as PNG, offer **lossless** compression.

Scene Noise

Lighting:

- **sunlight changes:** time of day, weather and season
- **varying artificial light sources:** streetlamps, headlights, etc.
- **interior light sources oscillate** with power supply frequency
- objects cast **shadows** *causing false image features*
- as objects move **shadows change.**



Scene Noise

Occlusion:

- objects are frequently obscured by other objects: **occlusion**
 - big problem for recognition systems in image understanding tasks
 - “you cannot see what you cannot see”
 - “you cannot know what you cannot see”.



Theoretical Noise Models

- When we have quantities that vary a lot and are quite unpredictable, we can use **theoretical models** to simulate and model their behaviour.
- The idea is that we can insert artificial noise in an image and then we can test the effectiveness of our denoising algorithms on that image.
- We hope that our algorithms will do an equally good job with real noise.

Types of Image Noise : **Salt & Pepper**

Salt & Pepper Noise (also known as *impulse* or *spike* noise):

- **random** white or black (high/low) value pixels into the image
- follows a binary high-low bi-modal noise distribution.
- causes abrupt variations

Example: “dead pixel” noise - when a pixel in the sensor (or display) fails to respond to input correctly.

- Less common in modern image sensors but present in cheap sensors (can effect single colour channels).



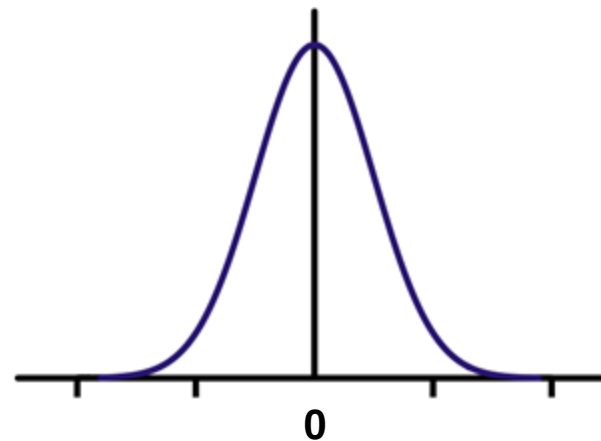
Types of Image Noise : **Gaussian**

Gaussian Noise (also known as *additive* noise):

- small random variation of the image signal around its true value following the **Gaussian distribution** (added *perturbation*)
- causes small variations
- The **most common** noise model in image processing
- describes effectively most types of random noise encountered in the image processing pipeline
 - especially sensor noise.

$$g(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{x^2}{2\sigma^2}}$$

(zero mean)



Q: How can we remove image noise?

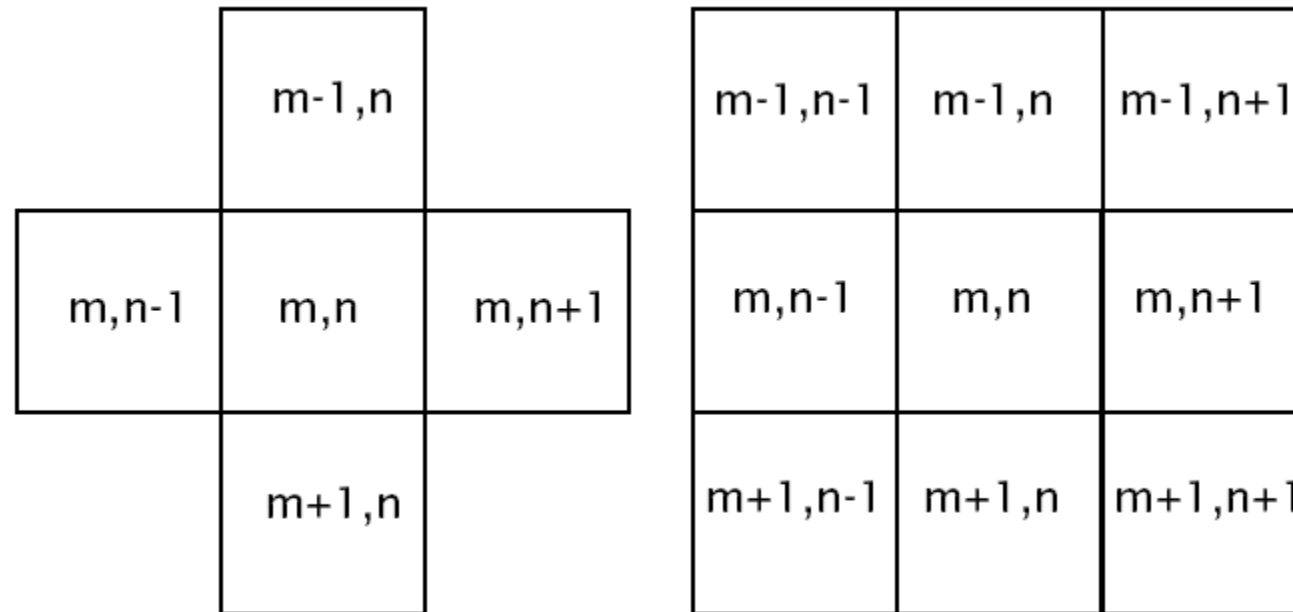
A: “filter it out” using a range of image filtering techniques

Concept of “filtering”: filter out (i.e. remove) the noise whilst retaining the information. (aka denoising)

... based on Pixel Neighbourhoods

Image Filtering and many other operations use $N \times N$ image neighbourhoods.

- local image neighbourhoods define **connectivity** or a local area of **influence, relevance or interest**.

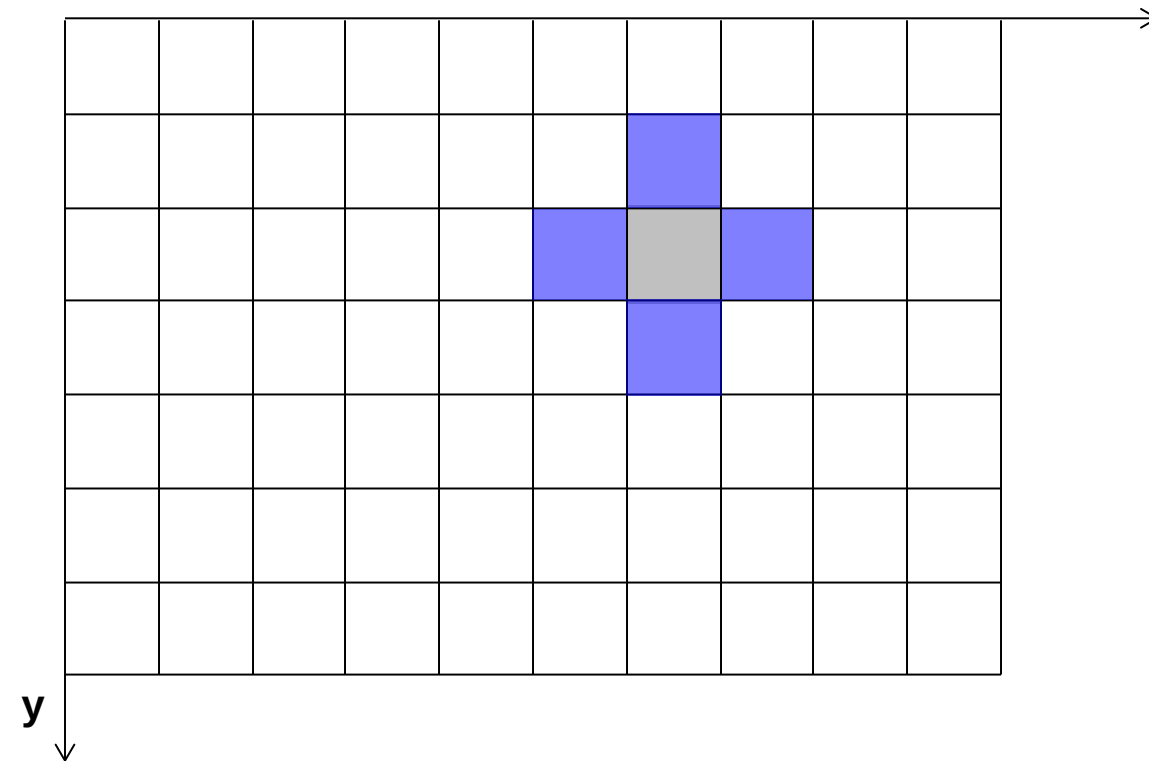


Connectivity choice : 4 or 8 connectivity

Neighbours of a Pixel

A pixel p at coordinates (x, y) has **four horizontal and vertical** neighbours (*4-neighbours* / $N_4(p)$), whose coordinates are given by:

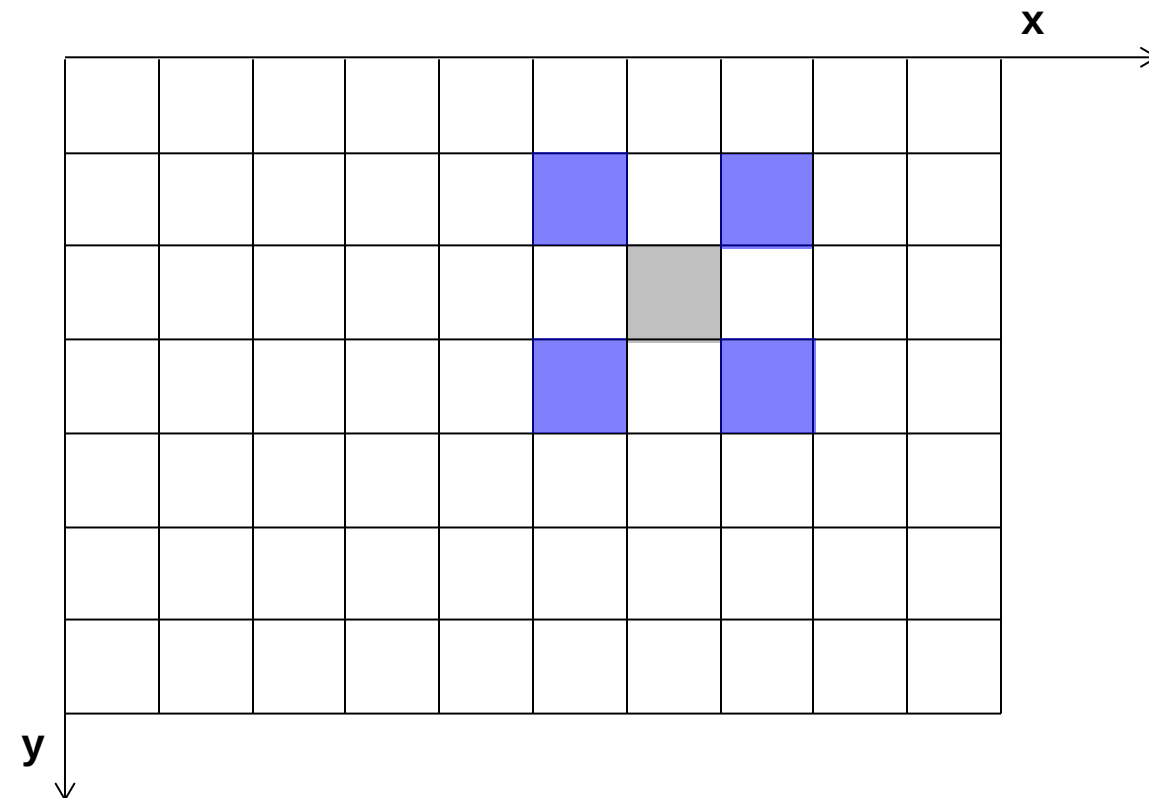
$$(x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)$$



Neighbours of a Pixel

A pixel p at coordinates (x, y) has **four diagonal** neighbours ($N_D(p)$), whose coordinates are given by:

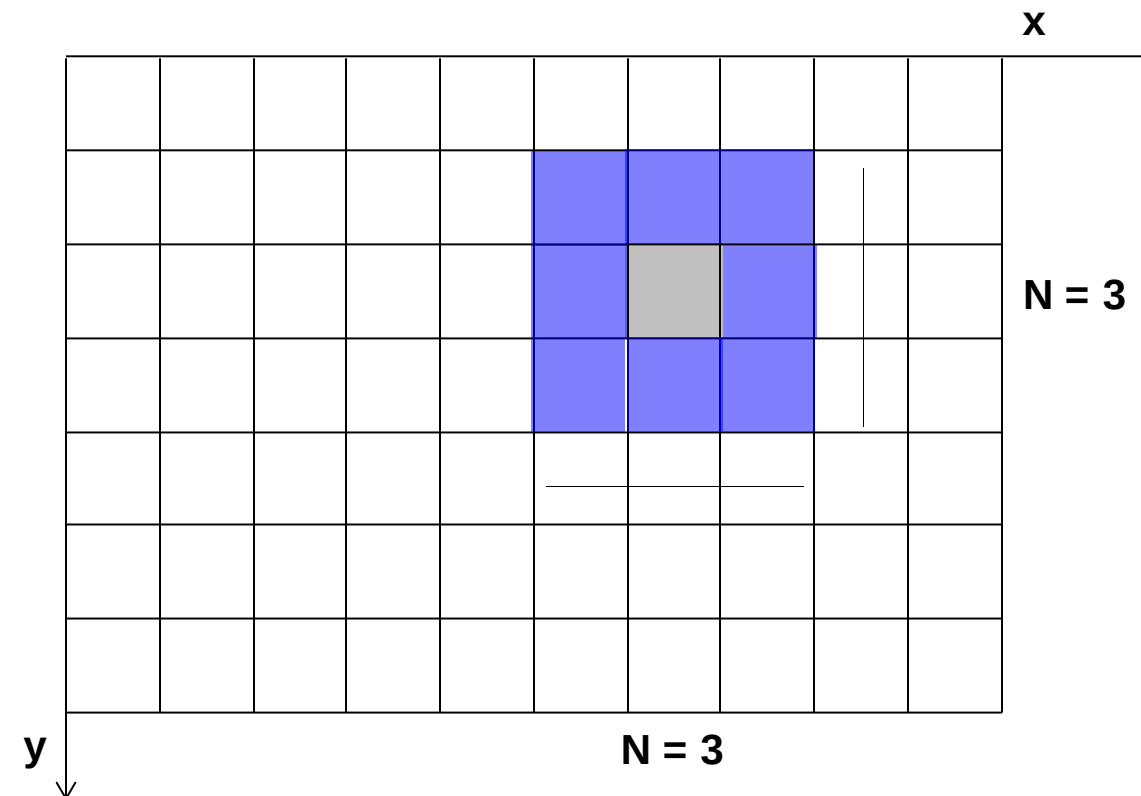
$$(x + 1, y + 1), (x + 1, y - 1), (x - 1, y + 1), (x - 1, y - 1)$$



Pixel Neighbourhood

Together they form the 3×3 local pixel neighbourhood (*8-neighbours* / $N_8(p)$).

- In general: $N \times N$ local pixel neighbourhoods.



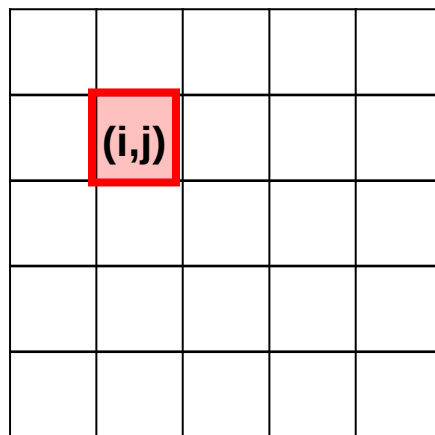
Spatial Filtering

Iterate through the pixels we want to process (perhaps the whole image).

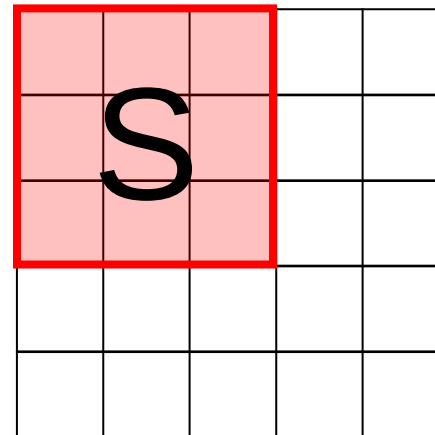
for each pixel (i,j) :

- consider a neighbourhood S of (i,j) . Usually (i,j) will be at the centre of S .
- process the pixel values of S to find a new value for pixel (i,j) .

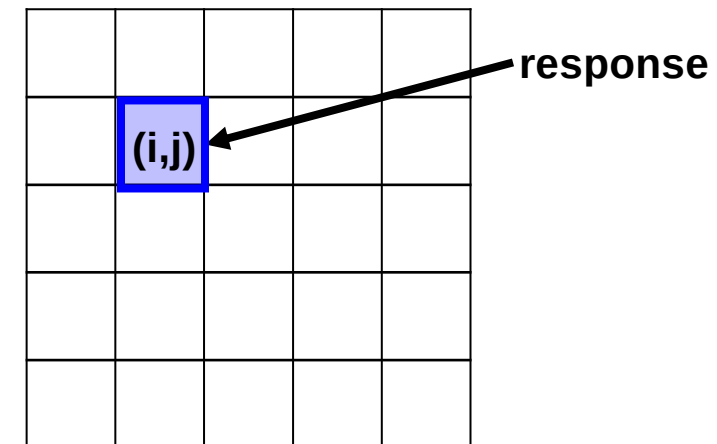
Replace the original pixel values with the new ones (called the filter's responses).



Original image



Neighbourhood of (i,j)
in the original image



Processed (output) image

We use **pixel neighbourhoods** with (either linear and non-linear) **spatial filtering** to **remove image noise**

- **Linear Filtering:** output pixel is a **linear** combination of the corresponding input pixel's neighbourhood.
- **Non-Linear Filtering:** output pixel is a **non-linear** function of the corresponding input pixel's neighbourhood. In practice, some decision-based algorithm is employed.

Spatial Non-linear Filters – *at a glance*

For an $N \times N$ image neighbourhood, N_{xy} , centred at pixel (x, y) indexed by (s, t) , the following simple statistical filters can be defined to replace each pixel with the min/max/median of the input neighbourhood:

Min $I_{output}(x, y) = \min_{(s,t) \in N_{xy}} \{I_{input}(s, t)\}$

Max $I_{output}(x, y) = \max_{(s,t) \in N_{xy}} \{I_{input}(s, t)\}$

Median $I_{output}(x, y) = \text{median}_{(s,t) \in N_{xy}} \{I_{input}(s, t)\}$

+ other variants (to follow)

Mean Filter

Operation: replace a given pixel with the mean (unweighted average) of its $N \times N$ image neighbourhood S :

- For $N = 3$, filter mask is

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

$$I_{output}(i, j) = \frac{1}{N^2} \sum_{(i, j) \in S} I_{input}(i, j)$$

N is odd.

Effect: eliminates sudden intensity jumps which could be caused by some noise processes, i.e. *eliminates large deviations from the norm*.

Mean Filter

Example: Suppose we have the 3 x 3 neighbourhood:

2	2	3
3	30	2
1	3	2

The value 30 is relatively large - noise *spike*.

Mean filtering will suppress it:

2	2	3
3	5.3	2
1	3	2

Mean Filter against Gaussian Noise



Image with Gaussian noise



mean (3x3)



mean (5x5)

Gaussian noise removed via $N \times N$ kernels

- Notice the loss of **high frequency** image detail.
- An edge enhancement technique could be used to improve this situation.

Mean Filter against Salt & Pepper Noise



Image with Salt & Pepper Noise



mean (3x3)



mean (5x5)

- The **large deviation** of the noise values from the distribution means that they influence the mean value significantly.
- Noise is still apparent in the filtered result.

Mean Filter against Noise



Image with Salt & Pepper Noise



mean (3x3)



mean (5x5)

- **Gaussian Noise:** distributed around the original value - the mean can easily correct for this.
- **Salt and Pepper Noise:** significant deviation from the local distribution.

Mean Filter : Drawbacks

- Mean filter **not robust to large noise deviations** (*i.e. statistical outliers*).
 - e.g. a single pixel with a very unrepresentative value.



Salt & pepper noise



mean (3x3)

- Mean filter **causes edge blurring**.
 - removes the high frequency sharp detail.



mean (5x5)

Mean Filter : Variants

Alpha trimmed mean:
$$I_{output}(x, y) = \frac{1}{N^2 - 2d} \sum_{(s,t) \in N_{xy}} I_{Input_r}(s, t)$$

The dimension of the neighbourhood N_{xy} is $N \times N$. The d lowest and the d highest intensity levels of the image in N_{xy} are deleted with $I_{Input_r}(s, t)$ denoting the remaining (reduced set of) $N^2 - 2d$ pixels in N_{xy} .

- Ignores excessively small/large values (outliers)

Harmonic mean:
$$I_{output}(x, y) = \frac{N^2}{\sum_{(s,t) \in N_{xy}} \frac{1}{I_{input}(s,t)}}$$

Mean Filter : Smoothing (low pass) Filter

- Mean filter could be used as a (very basic) general smoothing filter.
- It is spreading pixel values (as well as noise) on the neighbourhood of a pixel.



Original image



mean (3x3)



mean (7x7)

- 7 x 7 (spatially wider) filter achieved much more smoothing than 3 x 3.

Mean Filter : Smoothing (low pass) Filter



Original image



mean (3x3) applied 3 times



mean (7x7)

- Variation: 3 x 3 applied **three** times.
 - Spatial locality means less smoothing than single 7 x 7.
- Variation: **threshold-based filtering** – replace value if **difference > threshold**.

Median Filter

Operation: replace a given pixel with median of its $N \times N$ image neighbourhood.

Example: for 3×3 :

123	125	126	130	140
122	124	126	127	135
118	120	150	125	134
119	115	119	123	133
111	116	110	120	130

Neighbourhood values:

115, 119, 120, 123, 124,
125, 126, 127, 150

Median value: 124

Effect: eliminates sudden intensity jumps which could be caused by some noise processes, i.e. large deviations from the norm.

- It is **robust to statistical outliers** (unlike the mean filter).
- But it requires **sorting** pixel values!

Median Filter

Example: Suppose we have the 3 x 3 neighbourhood:

2	6	3
---	---	---

14	81	2
----	----	---

The value 81 is relatively large - noise spike (an outlier).

13	4	1
----	---	---

Median filtering will suppress it:

2	6	3
---	---	---

14	4	2
----	---	---

13	4	1
----	---	---

Median Filter

2	6	3		2	6	3
14	81	2		14	4	2
13	4	1		13	4	1

- Step 1: Convert 3×3 matrix to a **row vector** (2, 6, 3, 14, 81, 2, 13, 4, 1)
- **Sort** numbers as a sequence of increasing values (1, 2, 2, 3, 4, 6, 13, 14, 81)
- The **median** = the midpoint of the sorted distribution of values.
 - **fifth largest value**, which is 4.

Median Filter

requires a **sort** operation

- *Common method* : **Bubble Sort**

- $O(n^2)$ for n items
- $O(N^4)$ for an $N \times N$ neighbourhood.

- *Common improvement* : **Quicksort**

- $O(n \log_2 n)$ for n items
- $O(N^2 \log_2 N^2)$ for an $N \times N$ neighbourhood.

Median Filter against Gaussian Noise



Image with Gaussian noise

median (3x3)

- The Gaussian noise was removed with mild degradation in image quality.
- looks *posterised*, like a cartoon, because *fewer* values are used *more* often

Median filter against Gaussian noise



Even more Gaussian noise

median (3x3)

- Some of the Gaussian noise removed.
 - Limited degradation in image quality.



Image with Gaussian noise



mean (3x3)

Mean VS Median



Image with Gaussian noise



median (3x3)

Median Filter against Salt & Pepper Noise



Salt and pepper noise

median (3x3)

- The median filter is good at eliminating salt and pepper noise.
 - It overcomes statistical outliers.



Salt & pepper noise



mean (3x3)

Mean VS Median



Salt & pepper noise



median (3x3)

Application: Dead-Pixel Removal



Bad pixels, i.e. salt/pepper noise caused by camera sensor faults **filtered out**.



Bad pixels **not filtered out**.

Miricle thermal camera (expensive!) – with and without “dead pixel removal” turned on in the camera driver.

Conservative Smoothing

Operation: compare a pixel value to min and max of $N \times N$ neighbourhood:

- replace by *min* if $< min$
- replace by *max* if $> max$

Example: for $N = 3$

123	125	126	130	140
122	124	126	127	135
118	120	150	125	134
119	115	119	123	133
111	116	110	120	130

Neighborhood values:

115, 119, 120, 123, 124,
125, 126, 127, 150

Max: 127, Min: 115

Effect: eliminates sudden intensity jumps.

- It is called conservative because it does not change all pixels

Conservative Smoothing

123	125	126	130	140
122	124	126	127	135
118	120	150	125	134
119	115	119	123	133
111	116	110	120	130

Neighborhood values:

115, 119, 120, 123, 124,
125, 126, 127, 150

Max: 127, Min: 115

Here, 150 replaced by 127 (max of its 8 neighbours).

- A conservative approach to smoothing:
 - a pixel value will change only if it is outside the range of its neighbours.
 - only isolated outliers are eliminated.

Conservative Smoothing against Gaussian Noise



Gaussian noise



Conservative (3x3)

- Maximum image detail preserved at expense of noise suppression.

Conservative Smoothing against Salt & Pepper Noise



Image with salt & pepper noise



Conservative (3x3)

- Not all noise removed - multiple salt/pepper pixels in one neighbourhood.
 - One outlier will prevent another from being suppressed.

Conservative Smoothing against Salt & Pepper Noise

source: HIPR2

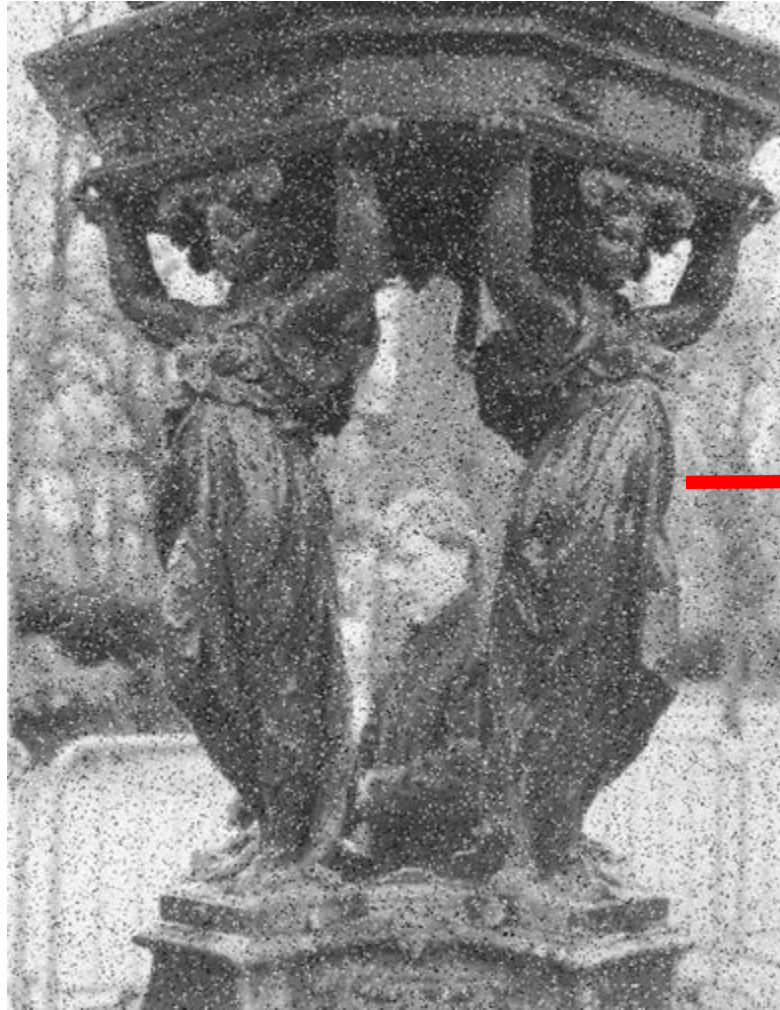
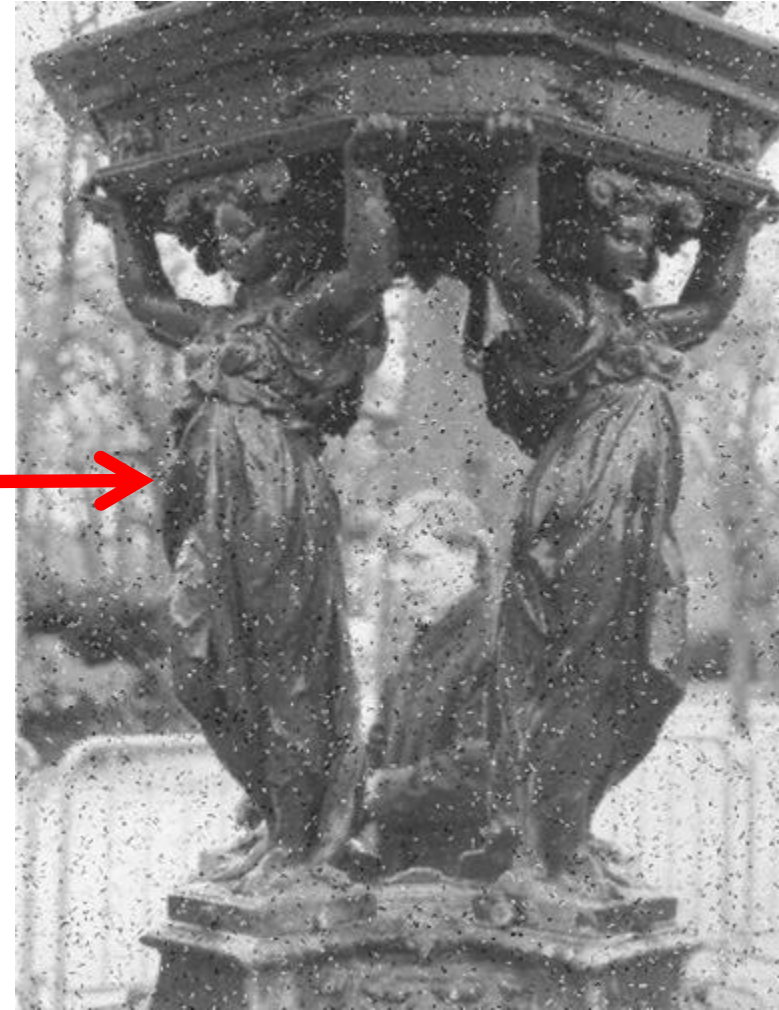


Image with salt & pepper noise



Conservative smoothing (3x3)



Salt & pepper noise



mean (3x3)



median (3x3)



conservative (3x3)

■ Salt & Pepper Comparison:

- Median performs best.
- Computationally most expensive



Gaussian noise



mean (3x3)



median (3x3)



conservative (3x3)

- **Gaussian Comparison:**

- Which is best???

Quick Demonstration

- Mean and Median Filtering for Gaussian Noise Removal:

https://github.com/atapour/ip-python-opencv/blob/main/gaussian_noise_removal.py



- Mean and Median Filtering for Salt and Pepper Noise Removal:

https://github.com/atapour/ip-python-opencv/blob/main/salt_pepper_filter.py



Summary

Image Noise

sources

common types of noise models



Image Neighbourhoods

key concept in image processing



Spatial Filters

mean, median & conservative

... the beginning of the filtering story